

imp

Q) why correctness of the algorithm is important? Define Loop invariant Property & prove the correctness of finding Summation of n numbers using Loop Invariant Property? [8]

→

1. why is correctness of an algorithm important?

- correctness of an algorithm means that the algorithm always produces the correct output for every valid input & terminates after a number of steps.

Importance -

- It guarantees accurate results
- It ensures reliability of software
- It avoids logical errors.
- It increases user confidence.
- It is essential in critical application such as banking, health care, aviation, & defence.

2) Loop invariant Property :-

- Def -
- A loop invariant is a condition that remains true before & after every iteration of the loop.
- It is used to prove the correctness of iteration algorithm.

A loop invariant proof consists of three steps:

1. initialization
2. maintenance
3. termination

3.1 - initialization

- It is true prior to the first iteration of the loop.

2. maintenance - If it is true before an iteration of the loop, it remains true before the next iteration.

3. termination - when the loop

termination gives us a useful property that helps show that the algorithm is correct.

- Importance of correctness -

- correctness is the binary property: an algorithm is either correct or it is not.
- proving correctness often involves mathematical induction or formal verification methods.

3. Algorithm for Summation of n numbers (1 mark) -
suppose the array is

$$A = \{a_1, a_2, a_3, \dots, a_n\}$$

Algorithm -

```

Sum(A, n)
sum ← 0
for i ← 1 to n
    sum ← sum + A[i]
return sum

```

4. flow diagram

Start

↓
read n & array

↓
sum = 0, i = 1

↓
i ≤ n?

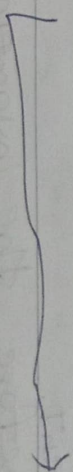
yes → no

Sum = sum + A[i]

Print sum

↓
stop

0
1 ← i + 1



5. Proof using loop invariant -

Loop invariant -
Before the i th iteration, Sum contains the sum of the first $(i-1)$ elements of the array.

Step 1: Initialization -

Before the first iteration -

$$\text{Sum} = 0$$

No elements have been added

therefore,

Sum = Sum of first 0 elements.

Hence, the loop invariant is true before the loop starts.

Step 2: maintenance -

Assume the invariant is true before iteration i .

that means -

$$\text{Sum} = A_1 + A_2 + \dots + A_{(i-1)}$$

During iteration -

$$\text{Sum} = \text{Sum} + A[i]$$

Now

$$\text{Sum} = A_1 + A_2 + \dots + A_{(i-1)} + A[i]$$

which is the sum of the first i element.

therefore, the invariant remains true after the iteration.

Step 3: termination -
the loop stops when

$$i = n + 1$$

According to the invariant,

$$\text{Sum} = A_1 + A_2 + \dots + A_n$$

which is the required total sum of all n numbers

Hence, the algorithm returns the correct result.

Ques

Q1) What is an iterative algorithm? explain iterative Algorithm Design issues using examples [1]

→

1) Definition of iterative Algorithm -

An iterative algorithm is an algorithm that solves a problem by repeating a sequence of instructions (such as for, while, or do-while) until a specified condition becomes false.

- state example -

finding the sum of first n natural numbers -

$$\text{Sum} \leftarrow 0$$

for $i \leftarrow 1$ to n

$$\text{Sum} \leftarrow \text{Sum} + i$$

Print (Sum)

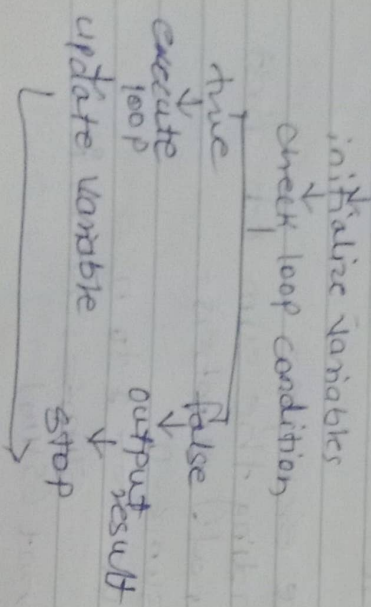
Here, the for loop repeatedly adds numbers from 1 to n , making it an iterative algorithm.

- State management:
An iterative algorithm often maintains a "state",

- termination - The loop must have a clear & guaranteed termination condition. A common error is an infinite loop, where the condition is never met.

- Maintain invariants: An invariant is a property that remains true throughout the loop.

2. General Structure of an Iterative Algorithm:



3. Iterative Algorithm Design Issues - while designing an iterative algorithm, the following issues must be considered.

Example: $\sum_{i=1}^n i$
Sum = 0

If Sum is not initialized, the result will be incorrect.

2. Loop Condition - the stopping condition must be correct.

ex- for $i = 1$ to n

If the condition is incorrect, the loop may execute too many or too few times.

3. Update Statement - the loop variable must be updated during each iteration.

ex $i = i + 1$

4. termination -

the algorithm should always stop after a finite number of iterations.

when i becomes greater than n , the loop terminates.

4. example: factorial using iteration.

```

FACTORIAL (n)
    fact ← 1
    for i ← 1 to n
        fact ← fact * i
    return fact
  
```

for $n = 5$

iteration	i	fact
start	-	1
1	1	1
2	2	2
3	3	6
4	4	24
5	5	120

output = 120

Advantages -

- faster execution than recursion
- uses less memory
- easy to debug
- suitable for large inputs.

Ques 3

(c) Write a short note on any four problem-solving strategies - [8-marks]

- problem force strategy
- 1. Brute force strategy
- definite

Brute force solves a problem by checking all possible soln & selecting the correct one.

Example -

- linear search
- selection sort

Advantages -

- easy to understand
- works for every problem.

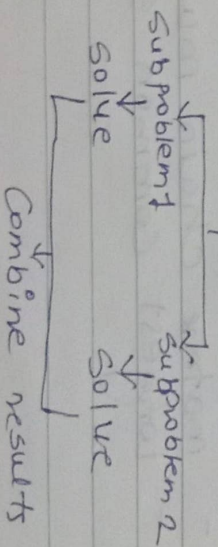
Disadvantages -

- slow for large inputs.

2. Divide and Conquer:

The whole problem is divided into two subproblems. if sub-problems are large then these are divided further into sub sub-problem.

Problem



Example

- merge sort
- quick sort
- Binary Search

Advantages

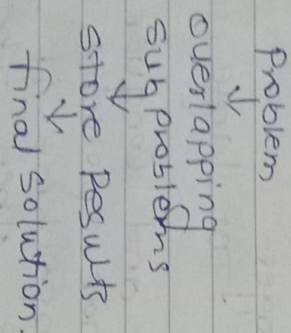
- faster execution
- efficient for large datasets

3. Dynamic programming :-

this method is used when we have to build up a solution to the problem via a sequence of intermediate steps.

Example : fibonacci series.

Diagram -



Example

- fibonacci series
- 0/1 Knapsack
- matrix chain multiplication
- longest common subsequence

Advantages -

- Avoids repeated calculations
- improve efficiency

4) Greedy Search -

Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious & immediate benefit.

Example -

- Prim's Algo
- Kruskal's Algo
- Dijkstra's Algo
- Huffman Coding

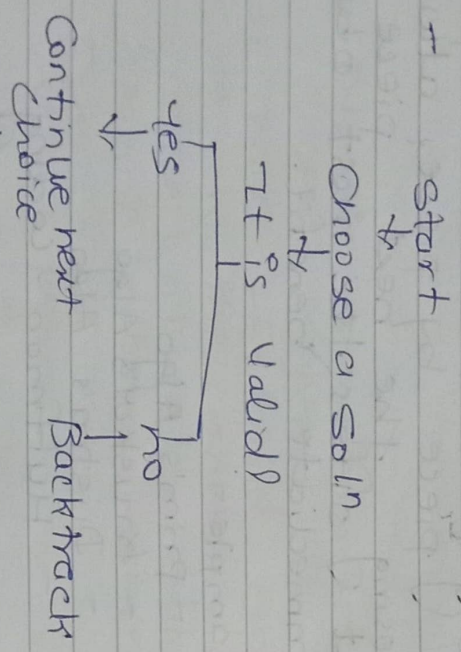
Advantages -

- simple
- fast

Limitations -

- Does not always produce the optimal solution.

5) Backtracking Strategy -
 - Backtracking is an algorithmic technique that builds step by step. If a partial soln cannot lead to a valid complete soln, it undoes the last step & tries another alternative.



Complete soln.

6) Recursive strategy -

A Recursive strategy is a problem-solving technique in which a function solves a problem by calling itself on smaller instances of the same problem until a base case is reached.

Structure of Recursion -
 A recursive algo
 1. Base Case - stops recursion
 2. recursive case - calls itself with a smaller problem.

Q) How to prove that an algorithm is correct? How to prove the of an algorithm using a counter? Give a suitable example.

1. Algorithm correctness?
 An algorithm is correct if it satisfies the following conditions:

1. Partial correctness: If the algorithm terminates, it produces the correct output.
2. termination - the algorithm always finishes after a finite number of steps.

Therefore,
 Correctness = Partial correctness + termination

2. methods to prove correctness -

the correctness of an algorithm can be proved using:

1. mathematical induction
2. loop invariant
3. counter example
4. proof by contradiction.

Among these, counter example is commonly used to show that an algorithm is incorrect.

3. Counter example -

A counter example is a specific input for which an algorithm produces an incorrect result.

If even one counterexample exists, the algorithm is not correct.

4. Example using Counter Example -

incorrect Algorithm -

Suppose we want to find the maximum of three numbers.

max(a, b, c)

if (a > b)

max = a

else

max = b

return max

Counter

$$a = 5$$

$$b = 8$$

$$c = 12$$

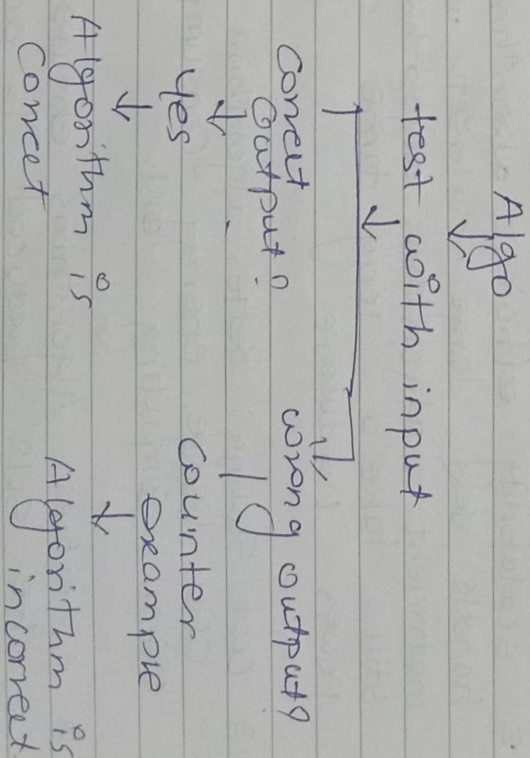
algorithm output:
8

correct output:

$$12$$

Since the Algorithm ignores c, it fails for this input Hence,

the algorithm is incorrect because a counter example exists.



(c) Given the fastest computer & hypothetically infinite memory, do we still need to study algorithms? justify.

→ yes, we still need to study algorithms.

Justification-

1. efficiency - A good algorithm solves problems faster by reducing the number of operations, even on very fast computers.

2. scalability: efficient algorithm solves problems faster by reducing the number of operations, even on very fast computers.

2. scalability - efficient algorithm can handle very large datasets, whereas inefficient algorithm algorithms may still take a long time despite power hardware.

3. Cost Saving: Better algorithms reduce CPU usage, energy consumption, & overall computing cost.

Correctness: Algorithms ensure that the soln produced is accurate & reliable

Examples-

To search an element in a sorted list of 1000,000 items:

• Linear Search: up to 100000 comparisons

• Binary Search: About 20 comparisons.
even with the fastest computer, Binary Search is much more efficient than linear search.

→ (c) How can we relate algorithms to technology? Briefly explain.

An algorithm is a step-by-step procedure to solve a problem. modern technology depends on efficient algorithms to provide fast, accurate & reliable soln. therefore, algorithms are considered a core technology in computing.

- Relation of Algorithms to technology

1. improves performs -

- efficient algorithms make software run faster & use fewer resources.

- merge sort is faster than bubble sort for large datasets

2. reduces time & memory -

- Good algorithms require less

execution time & memory.

- this improves the overall performance of computer systems.

3. Handles large data -

- modern applications process huge amounts of data using efficient algo.
- ex: search engines like Google use advanced search algorithms.

4. Supports modern technologies -

- Algorithms are the foundation of technologies such as:
 - Artificial intelligence (AI)
 - ML
 - Blockchain
 - CC
 - cybersecurity

5. Cost-effective:

- efficient algorithms reduce CPU usage, power consumption, & hardware costs.

6. Solves real-world problems -

- algorithms used in many real-life applications:
 - GPS navigation
 - online banking
 - e-commerce recommendations
 - social media feeds.

- Algorithms are closely related to technology because they improve the speed, efficiency, accuracy, & reliability.

Q1)

Consider an array A of n integers which are already in sorted order, let x be the number being searched in the array A in a linear fashion. The code fragment performing this task is given below:

```
int lin_search (int A[])
{
    i = 0; flag = 0;
    do { if (x == A[i]) then
        return (i); // number found
    } else
    { while (i++ < n)
        return (0); // NO. not found
    }
}
```

i) Is this code fragment efficient? (we wish to use linear search only) justify your answers.

ii) Does it attribute to any design issue with respect to iterative algo.

Briefly explain.

→ i) Yes the code is efficient for linear search, but it can be improved because the array is already sorted.

Justification:

1. Best case: If the element x is at the first position, only 1 comparison is required.
- time complexity = $O(1)$

2. worst case: If x is at the last position or not present, all n elements are checked.
- time complexity = $O(n)$

3. since the array is already sorted, the algorithm can be made more efficient by stopping the search when $AC[i] > x$, because x cannot appear after that point.

4. therefore, although the given algorithm correctly performs linear search, it does not take advantage of the sorted order of the array.

ii) Does it attribute to any design issue respect to it does not use the information that the array is sorted.

Design issues:

1. unnecessary iterations.
- The loop continues even after reaching an element greater than x .

2. Poor loop optimization:

- The stopping condition considers only $i < n$.
- It should also stop when $AC[i] > x$.

3. Reduced efficiency:

- Extra comparison increases execution time, especially for large arrays.

improved condition:

while $C[i] \leq x$

or

if $AC[i] > x$

break;

Q) what is an iterative Algo? Explain iterative algorithm design issues using suitable example. 8

→

- An iterative algorithm is an algorithm that repeats a set of instructions using loops (for, while, or do-while) until a specified condition becomes false.

- Instead of using recursion, It performs repeated execution through iteration.

Example -

```
Algorithm to print Nos from 1 to 5
{
  for  $i=1, i \leq 5; i++$ 
  {
    Print( $i$ );
  }
```

output:

1 2 3 4 5

This is an iterative algorithm because the loop executes repeatedly.

Iterative Algorithm Design issues -

while designing an iterative algorithm, the following issues should be considered.

1. Loop initialization -

- loop variables must be initialized correctly before the loop starts.

- incorrect initialization may produce wrong results.

Example

Correct:

```
 $i=0;$ 
```

```
while ( $i \leq 5$ )
```

wrong:

```
 $i=5$ 
```

```
while ( $i \leq 5$ )
```

2. Loop termination -

- the loop must stop after a finite number of iterations
- incorrect termination may cause an infinite loop.

Example -

Correct:

```
while ( $i < n$ )
```

wrong 'while ($i >= 0$)

(if i is never decreased, the loop runs forever.)

3. Loop update

- the loop variable must be updated in every iteration.

- otherwise, the algorithm may never terminate.

eg -

Correct

```
 $i++;$ 
```

wrong - // missing $i++$

4) Avoid unnecessary iterations -

- the algorithm should stop as soon as the required result is obtained.

- this improves efficiency.

eg - in a sorted array, stop search when $AC[i] > x$.

```
 $i++$   
if ( $AC[i] > x$ )  
break;
```


5) Correctness of output -

- the algorithm should produce the correct result for every valid input.
- the using different test cases.

ex -

- find the maximum element in an array & verify the result for:

- positive numbers
- negative numbers
- single-element array

6. Time & Space Efficient :-

- Reduce unnecessary computations
- use minimum memory whenever possible.
- using one variable to calculate the sum instead of storing intermediate results.

Advantages of iterative algo.

- simple to understand
- faster than recursion in many cases.
- uses less memory
- easy to debug & implement.

Proof

Q) Consider the following algorithm to

find the square of a number.

Prove the correctness of this algorithm using the principle of mathematical induction. [7]

Given

int sqrl(int n)

{

if (n == 0)

return 0;

else

return (2*n + sqrl(n-1) - 1);

}

We need to prove that the algorithm returns:

$$\text{sqrl}(n) = n^2$$

using mathematical induction.

Proof by mathematical induction -

Step 1: Base Case

for $n = 0$

from $n = 0$ the algo

$$\text{sqrl}(0) = 0$$

mathematically,

$$0^2 = 0$$

Hence,

$$\text{sqrl}(0) = 0^2$$

Base Case is true.

Step 2: induction Hypothesis -

Assume the algo is correct for $n = k$.

that is,

$$\text{sq}(k) = k^2$$

this assumption is called the induction Hypothesis.

Step 3: induction step

Now prove for $n = k+1$.
from the algo.

$$\text{sq}(k+1)$$

$$= 2(k+1) + \text{sq}(k) - 1$$

Using the induction hypothesis,

$$= 2(k+1) + k^2 - 1$$

simplify,

$$= 2k + 2 + k^2 - 1$$

$$= k^2 + 2k + 1$$

$$= (k+1)^2$$

thus,

$$\text{sq}(k+1) = (k+1)^2$$

Hence, the statement is true for $k+1$.

impl

Q1

Prove the correctness of the following algorithm, which finds the minimum number from the given array of n numbers.

int min-element (int A[])

{

min = A[0];

for ($i = 1; i < n; i++$)

{

if (A[i] < min)

min = A[i];

}

return (min);

}

→ proof of correctness.

Step 1: loop invariant

loop invariant:

Before each iteration of the loop, min contains the smallest element among A[0] to A[i-1].

Step 2: initialization

Before the loop starts:

$$\text{min} = A[0]$$

only the first element has been considered.

therefore,

$$\text{min} = \text{smallest element in } A[0]$$

Hence, the loop invariant is true before the first iteration.

Step 3: maintenance

Assume the loop invariant is true before the current iteration.

that means:

$\min = \text{smallest element among } A[0] \dots A[i-1]$

Now Compare:

$A[i] < \min$

Case 1:

- If true,

$\min = A[i]$

Now $\min$ becomes the smallest element among $A[0] \dots A[i]$.

Case 2:

- If false,

$\min$ remains unchanged.

$\exists$ it's still the smallest element among $A[0] \dots A[i]$.

Step 4: termination.

the loop stops when,

$i = n$
At this point, all array elements have been checked.

According to the loop invariant,
 $\min$ of smallest element among $A[0] \dots A[n-1]$

the algorithm returns -

return $\min$

which is the minimum element of the entire array.

Q)

Prove the correctness of the following algorithm, which finds the factorial of a given No.

→

Given

int fact(int n)

{

if ($n = 0$)

return 1;

else

return ($n \cdot \text{fact}(n-1)$);

}

→

Proof by mathematical induction -
we need to prove that the algorithm returns:

$\text{fact}(n) = n!$

for every non-negative integer n .

Step 1: Base case

for $n = 0$

from the algorithm,

$\text{fact}(0) = 1$

mathematically,

$0! = 1$

hence,

$\text{fact}(0) = 0!$

Base case is true

Step 2: induction Hypothesis -

Assume the algorithm is correct

for $n = k$.

that is,

$$\text{fact}(k) = k!$$

This assumption is called the induction hypothesis.

Step 3: induction Step

Now prove for $n = k+1$.

from the algorithm,

$$\text{fact}(k+1)$$

$$= (k+1) \times \text{fact}(k)$$

using the induction hypothesis,

$$= (k+1) \times k!$$

Since,

$$(k+1)! = (k+1) \times k!$$

therefore,

$$(k+1)! = (k+1) \times k!$$

$$\text{fact}(k+1) = (k+1)!.$$

Thus, the statement is true for $k+1$.

Step 4: conclusion

Since

- the base case is true, &

- if true k , then true for $k+1$

$\therefore \text{fact}(n) = n!$ for all $n \geq 0$.

ex -

for $n = 4$.

$\text{fact}(4)$

$$= 4 \times \text{fact}(3)$$

$$= 4 \times 3 \times \text{fact}(2)$$

$$= 4 \times 3 \times 2 \times \text{fact}(1)$$

$$= 4 \times 3 \times 2 \times 1 \times \text{fact}(0)$$

$$= 4 \times 3 \times 2 \times 1$$

$$= 24$$

$$\underline{\underline{4! = 24}}$$

unit 2

Q) what do you understand by Best Case, worst case, worst case & Average Case behavior of an algorithm? is Average case efficiency an average of best-case & worst-case efficiencies? justify your answer.

→ The behaviour of an algorithm refers to the amount of time or memory it requires to execute for different input cases.

there are three types of analysis:

- Best Case
- worst case
- Average case

1. Base Case -

- The best case is the minimum taken by an algorithm.
- It occurs when the input is in the most favourable condition.
ex- linear search

searching 10 in:

[10, 20, 30, 40, 50]

The element is found at the first position.

Time complexity: $O(1)$

2. worst case -

- the case worst case is the maximum time taken by an algorithm.
- It occurs when the input is in the least favourable condition.

eg. Linear Search,

Searching 50 or an element not present:

[10, 20, 30, 40, 50]

All elements are checked.

time complexity: $O(n)$

3 Average case

- the average case is the expected time taken over all possible inputs.
- It average case is the expected time taken over all possible inputs
- It represents the algorithm's average performance.

eg. Linear search

If the element can appear at any position with equal probability, the average number of comparisons is:

$$\frac{n+1}{2}$$

time com = $O(n)$

Difference.

Best Case	Average Case	Worst Case
- minimum execution time	- expected execution time	- maximum execution time
- most favorable input	- average of all possible inputs	- least favorable input
- $O(1)$ (Linear Search)	- $O(n)$	- $O(n)$

→ FS Average case efficiency the Average of Best-case & worst-case efficiencies. NO.

Average case is not arithmetic average of the best-case & worst-case efficiencies. justification -

- Average-case analysis considers all possible inputs & their probabilities.
- Best case Consider only the most favorable input.
- worst case considers only the least favourable input.

Example (Linear Search):

- Best case = 1 comparison
- worst case = n comparison
- Average case = $(n+1)/2$ comparisons

clearly,

$$\frac{1+n}{2}$$

is obtained from the expected No. of comparisons over all positions, not simply by averaging the complexities $O(1)$ & $O(n)$.

- Q2) / comment on statement - Best Case analysis of an algorithm may not give a clear idea of performance.

yes, the statement is true.

Reason -

- Best Case analysis shows the minimum execution time of an algorithm, which occurs under the most favorable input conditions.

- In practice, such ideal conditions do not occur every time.
- therefore, best case does not represent the actual or expected performance of the algorithm.

- worst case or average case: Analysis gives a more realistic measure of an algorithm's performance.

Example:

for linear search:

- Best Case: element is found at the first position $\rightarrow O(1)$.
- worst case: element is at the last position or not present $\rightarrow O(n)$

Hence, considering only the best case can give a misleading idea about the algorithm's overall performance.

- Q2) Analyze the given sorting algorithm for Best, Average & worst.

void sort(int arr[], int n)

```
int i, j, key;
for (i = 0; i < n; i++)
```

```
{
    j = i - 1;
```

```
    key = arr[i];
```

```
    while (j >= 0 & arr[j] > key)
```

```
    {
        arr[i+1] = arr[j];
```

```
        j = j - 1;
```

```
    }
    arr[i+1] = key;
}
```

```
}
```


1. Best Case Analysis -
Condition -
the array is already sorted in ascending order.

ex: 10 20 30 40 50

Analysis -
- the condition $alist[j] < key$ is false in every iteration.
- the while loop executes 0 times
- only one comparison is made for each element.

- total comparisons: $n-1$
time complexity

2. worst case analysis -
condition
the array is in descending order
example: 50 40 30 20 10

analysis -
each new element is compared with all previously sorted elements.

comparisons:
 $1 + 2 + 3 + \dots + (n-1)$

$$= \frac{n(n-1)}{2}$$

time complexity $O(n)^2$

3) Average Case Analysis -
condition -
The elements are in random order.

Analysis -
on average, each element is compared with half of the previous elements.
comparisons:

$$(n-1)/2 + (n-2)/2 + \dots + 1/2$$

approximately,

$$\frac{n(n-1)}{4}$$

time complexity $O(n)^2$

Case	input condition	comparisons	time complexity
Best Case	already	$n-1$	$O(n)$
Average	Random	$\frac{n(n-1)}{4}$	$O(n)^2$
worst	Random	$\frac{n(n-1)}{2}$	$O(n)^2$

Q) What is Best, Average & worst Case Analysis of algorithms?

analyse the following linear search algorithm.

```
int linear-search (int a[], int n,
int item)
```

```
{
    int i;
```

```
    for (i=0; i<n; i++)
```

```
        if (a[i] == item)
```

```
            return a[i];
```

```
    return -1;
```

```
}
```

→ Analysis of linear search

1. Best Case Analysis -

Condition -

the element is present at the first position.

example:

array: 10 20 30 40 50

item: 10

Comparisons: 1

time complexity $O(1)$

2) Average Case Analysis

condition

the element may be present at any position with equal probability

example:

array: 10 20 30 40 50

item: 30

Average comparisons:

$$\frac{n+1}{2}$$

time complexity

$$O(n)$$

3. worst Case Analysis

condition

The element is at the last position or not present in the array.

example:

array: 10 20 30 40 50

item: 60

All n elements are compared.

Comparisons: n

time complexity

$$O(n)$$

Q) Consider the selection sort algorithm. Analyse the complexity of this algorithm. clearly indicate the assumptions made, if any.

Assumptions -

- let n be the number of elements in the array.
- Comparison ($A[i] < \min_val$) is the dominant operation.
- swapping takes constant time $O(1)$
- we count only the number of comparisons to analyze complexity.

Complexity Analysis -

1. outer loop

the outer loop executes:

for $i = 1; i < n-1; i++$

Number of iterations:

$n-1$

2. inner loop

for each value of i , the inner loop compares the remaining elements.

Number of comparisons:

for $i = 1 \rightarrow n-1$ comparisons

for $i = n-2$ comparisons

for $i = n-3 \rightarrow n-3$ comparisons.

for last pass $\rightarrow 1$ comparison
total comparisons:

$$(n-1) + (n-2) + (n-3) + \dots + 1$$

$$= \frac{n(n-1)}{2}$$

3) time complexity

Since every element is compared with the remaining unsorted elements regardless of the input order, the number of comparisons remains the same.

Best Case -

Already sorted array
comparisons:

$$\frac{n(n-1)}{2}$$

time complexity: $O(n^2)$

average case

random order array

$$\text{comparisons } \frac{n(n-1)}{2}$$

TC -

$$O(n^2)$$

Worst Case -
Reverse Sorted array.

Comparisons:

$$\frac{n(n-1)}{2}$$

$$TC: O(n^2)$$

Space Complexity -

Selection sort uses only one temporary variable for swapping.
 $O(1)$

Q) What do Ω & Θ notations mean?
When do we use Θ notation.

Asymptotic Notations -

Asymptotic notations are used to describe the growth rate of an algorithm's time or space complexity as the input size n increases.

1. Big-Omega (Ω) Notation -

Big-Omega (Ω) represents the lower bound of an algorithm.

It indicates the minimum time required by an algorithm.

Mathematical Definition -

$$f(n) = \Omega(g(n))$$

if there exist constants $c > 0$ & n_0 such that

$$f(n) \geq c \cdot g(n), \forall n \geq n_0$$

example

Linear Search:

- element found at the first position.
time complexity:

2. Big-theta (Θ) Notation

- Definition

- Big-theta (Θ) represents the tight bound of an algorithm.

- It gives both the upper bound & lower bound, showing the exact growth rate.

Mathematical Definition -

$$f(n) = \Theta(g(n))$$

if there exist position constants C_1, C_2 & n_0 such that

$$C_1 g(n) \leq f(n) \leq C_2 g(n), \forall n \geq n_0$$

Now check the reverse:
 $n^2 \leq C \cdot n$

Ho, because n^2 grows much faster than n , there is no constant C such that

$$n^2 \leq C \cdot n$$

for all sufficiently large n .

Hence,

$$n^2 \neq O(n)$$

Conclusion
therefore

$$f(n) = O(g(n)) \neq g(n) = O(f(n))$$

The implication is false in general because Big-O represents only upper bound of a function.

Q) If an algorithm has a time complexity of both $O(f(n))$ & $\Omega(g(n))$ for the same function $f(n)$ & $g(n)$, then what does it imply? explain.

→ In an algorithm has both $O(f(n))$ & $\Omega(g(n))$, & $f(n) = g(n)$ then the algorithm has a Θ (theta) time complexity.

Explanation -

- $O(f(n))$ gives the upper bound.
- $\Omega(g(n))$ gives the lower bound.
- If both bounds are the same function, then then algorithm has an exact bound, represented by Θ .

Mathematical representation -

i-f

$$T(n) = O(f(n))$$

&

$$T(n) = \Omega(f(n))$$

then,

$$T(n) = \Theta(f(n))$$

example

merge sort

- Upper Bound $\geq O(n \log n)$
- Lower Bound $= \Omega(n \log n)$

therefore,

$$\Theta(n \log n)$$

Q1) Is $\Theta(n^3) = O(n^3)$? justify your Ans.
 Ans - No, $\Theta(n^3)$ is not $O(n^2)$.

justification

By definition,

$$f(n) = O(g(n))$$

If there exist positive constants C_1, C_2 & no such that

$$C_1 g(n) \leq f(n) \leq C_2 g(n), \forall n \geq n_0$$

Here,

$$f(n) = \Theta(n^3), \quad g(n) = n^2$$

Consider the ratio:

$$\frac{\Theta(n^3)}{n^2} = \Theta(n)$$

$$\text{As } n \rightarrow \infty$$

$$\Theta(n) \rightarrow \infty$$

This means the ratio is not bounded by a constant. therefore $\Theta(n^3)$ grows much faster than n^2 .

$$\text{Hence, } \Theta(n^3) \neq O(n^2)$$

In fact,

$$\Theta(n^3) = O(n^3)$$

Since $\Theta(n^3)$ grows faster than n^2 , It is not $O(n^2)$, the correct asymptotic notation is: $\Theta(n^3) = O(n^3)$.

Q2) Explain the 3-SAT problem using an example. why is SAT so important in theoretical computer science?

1. what is SAT?

The Boolean satisfiability problem (SAT) is the problem of determining there exists an assignment of true (T) or false (F) values to variables such that the given Boolean expression becomes true.

2. what is 3-SAT?

The 3-SAT (3-satisfiability) problem is a special case of SAT in which each clause contains exactly three literals.

A 3-SAT formula is the AND (AND) of several clauses, & each clause is the OR (OR) of exactly three literals.

General form:

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge \dots$$

example

Consider the Boolean formula:

$$(x_1 \vee y_1 \vee z_1) \wedge (\neg x_1 \vee y_2 \vee z_2)$$

Assign the values:

$x = \text{false}$

$y = \text{true}$

$z = \text{false}$

then,

first clause:

$(\text{false} \vee \text{true} \vee \text{false}) = \text{true}$

second clause:

$(\text{true} \vee \text{true} \vee \text{false}) = \text{true}$

therefore,

$\text{True AND true} = \text{True}$

Hence, the formula is satisfiable, so this is a yes instance of the 3-SAT Problem.

Why is SAT important in theoretical Computer Science?

1. SAT was the first problem proved to be NP-complete.
2. It is used to determine whether a Boolean formula can be satisfied.
3. Many difficult computational problems can be converted to SAT.

e. SAT is widely used in:

- AI
- Circuit Design & Verification.
- Software testing
- Scheduling problems.
- Cryptography

\$\therefore\$ If an efficient algorithm is found for SAT, then all NP problem can be solved efficiently (P = NP).

Q)

What is polynomial-time Reducibility?

What is its important in computational complexity theory?

\$\rightarrow\$ Polynomial-time Reducibility is a method of converting one problem into another in polynomial time.

If a problem A can be transformed into problem B in polynomial time, where then A is polynomial-time reducible to B, denoted by:

$$A \leq_p B$$

This means that if problem B can be solved efficiently, then problem A can also be solved efficiently.

Example:

consider

- Problem A: 3-SAT
- Problem B: Vertex Cover

3-SAT can be transformed into vertex cover in polynomial time. therefore,

$$3\text{-SAT} \leq_p \text{Vertex Cover}$$

If we have a polynomial-time algorithm for vertex cover, then we can also 3-SAT efficiently.

Importance of polynomial-time reducibility:

1. Used to compare the difficulty of computational problem.
2. Helps prove that a problem is NP-complete.
3. Shows that one problem can be transformed into another in polynomial time.
4. It is a fundamental tool in computational complexity theory for classifying problems into P, NP, NP-complete, & NP-hard classes.

Diagram -

